

```
1 class RecurrenceRelation {
```

```
2 /**
```

```
3 * Definition: An equation or inequality that describes  
4 * a function in terms of its value on smaller inputs.  
5 */
```

```
6  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102  
103  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119  
120  
121  
122  
123  
124  
125  
126  
127  
128  
129  
130  
131  
132  
133  
134  
135  
136  
137  
138  
139  
140  
141  
142  
143  
144  
145  
146  
147  
148  
149  
150  
151  
152  
153  
154  
155  
156  
157  
158  
159  
160  
161  
162  
163  
164  
165  
166  
167  
168  
169  
170  
171  
172  
173  
174  
175  
176  
177  
178  
179  
180  
181  
182  
183  
184  
185  
186  
187  
188  
189  
190  
191  
192  
193  
194  
195  
196  
197  
198  
199  
200  
201  
202  
203  
204  
205  
206  
207  
208  
209  
210  
211  
212  
213  
214  
215  
216  
217  
218  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000
```

```
const concept = "Self-Similarity";  
let usage = ["Algorithm Analysis", "Time Complexity"];
```

```
// Analogy: Russian Nesting Dolls
```



```
}
```

Why are Recurrences Important?

// Foundation of Divide-and-Conquer Analysis

1. Divide

Break the problem into smaller subproblems (instances of the same problem).



2. Conquer

Recursively solve subproblems. If small enough, solve directly (Base Case).



3. Combine

Merge the solutions of subproblems to solve the original problem.

Example: Merge Sort Running Time

$$T(n) = 2T(n/2) + O(n)$$

// 2 subproblems of size n/2 + linear time to merge

The Anatomy of a Recurrence

The diagram shows the recurrence relation $T(n) = 2T(n/2) + n$ with three annotations in dashed boxes connected by lines to specific parts of the equation:

- A box labeled "Number of subproblems" points to the coefficient 2 in $2T(n/2)$.
- A box labeled "Cost of divide & combine steps" points to the n in $n/2$.
- A box labeled "Recursive call on smaller input" points to the T in $T(n/2)$.

$$T(n) = 2T(n/2) + n$$

The Base Case

The condition that stops the recursion. Typically for small n (e.g., $n=1$), the problem is solved directly in $O(1)$ time.

Methods for Solving Recurrences

```
function substitution() {  
    step1 = "Guess";  
    step2 = "Verify  
(Induction)";  
}
```

The "Guess & Prove" Approach

```
// Most general method.  
// Requires a good initial  
guess.  
// Used to prove  $O$  and  $\Omega$   
bounds.
```

```
function recursionTree() {  
    visualize("Tree");  
    return  
    sum(costs_per_level);  
}
```

The Visual Approach

```
// Models the flow of  
recursion.  
// Sums costs at each depth.  
// Great for generating  
guesses.
```

```
function masterMethod() {  
    if (form == "aT(n/b) +  
f(n)")  
        return lookup(Case);  
}
```

The "Cookbook" Approach

```
// Quickest for standard  
forms.  
// Relies on 3 specific cases.  
// Compares  $n^{(\log_b a)}$  vs  
f(n).
```

Substitution Method

// The "Guess and Prove" Technique



function make_guess()

Characterize the form of the solution based on experience or heuristics (e.g., Recursion Tree).
`guess = "O(n log n)"`



function verify_induction()

Use **Mathematical Induction** to find the constants `c` and `n0` and prove the guess is correct.
Show that $T(n) \leq c \cdot f(n)$.

Warning: This method is powerful but requires a good initial guess. If the guess is wrong, the induction will fail.

Substitution Method: Simple Example

// Solving $T(n) = T(n-1) + 1$

Recurrence: $T(n) = T(n-1) + 1$ Base Case: $T(1) = 1$

Step 1: Guess

We guess $T(n) = O(n)$

Goal: Prove $T(n) \leq cn$

Step 2: Inductive Step

Assume $T(n-1) \leq c(n-1)$

$$T(n) = T(n-1) + 1$$

$$T(n) \leq c(n-1) + 1$$

$$T(n) = cn - c + 1$$

We need: $cn - c + 1 \leq cn$

$$c \geq 1$$

Step 3: Base Case

For $n = 1$:

$$T(1) = 1$$

True if $c \geq 1$

✓ Proof holds for $c \geq 1$. Thus, $T(n) = O(n)$.

Substitution: Complex Example

$$\text{Target: } T(n) = 2T(n/2) + n$$

💡 1. The Guess

$$T(n) = O(n \lg n)$$

✅ 2. Inductive Hypothesis

Assume for $k < n$:

$$T(k) \leq c \cdot k \lg k$$

Specifically for $n/2$:

$$T(n/2) \leq c(n/2) \lg(n/2)$$

✓ Proven for $c \geq 1$

$$\text{01 } T(n) = 2T(n/2) + n \quad // \text{Original Recurrence}$$

$$\text{02 } \leq 2[c(n/2) \lg(n/2)] + n \quad // \text{Substitute Hypothesis}$$

$$\text{03 } = cn(\lg n - \lg 2) + n \quad // \text{Simplify: } \lg(a/b) = \lg a - \lg b$$

$$\text{04 } = cn \lg n - cn + n \quad // \text{Expand terms } (\lg 2 = 1)$$

$$\text{05 } = cn \lg n - (c - 1)n \quad // \text{Group residual terms}$$

$$\text{06 } \leq cn \lg n \quad // \text{True if } (c - 1)n \geq 0 \Rightarrow c \geq 1$$

Recursion-Tree Method

// Visualizing the Cost

1. Visualize

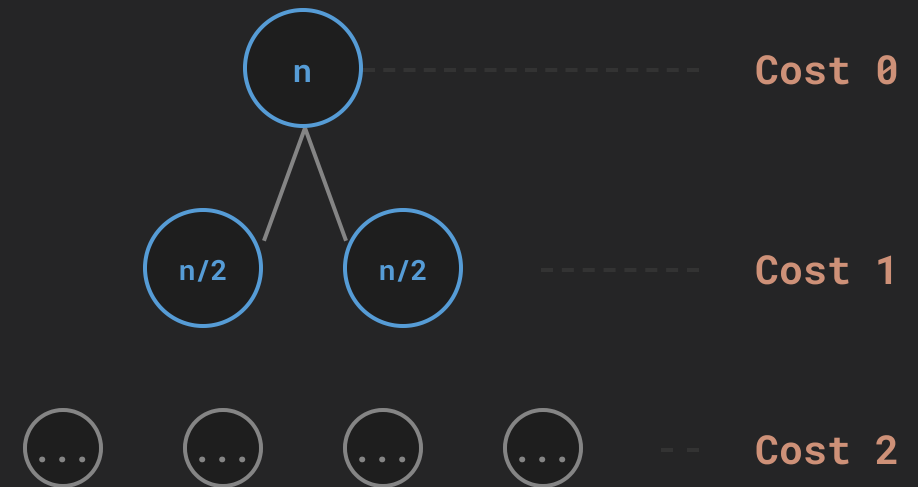
Draw a tree where each node represents the cost of a single subproblem.

2. Sum Levels

Sum the costs of all nodes at each depth level to get per-level costs.

3. Total

Sum all per-level costs to determine the total running time.



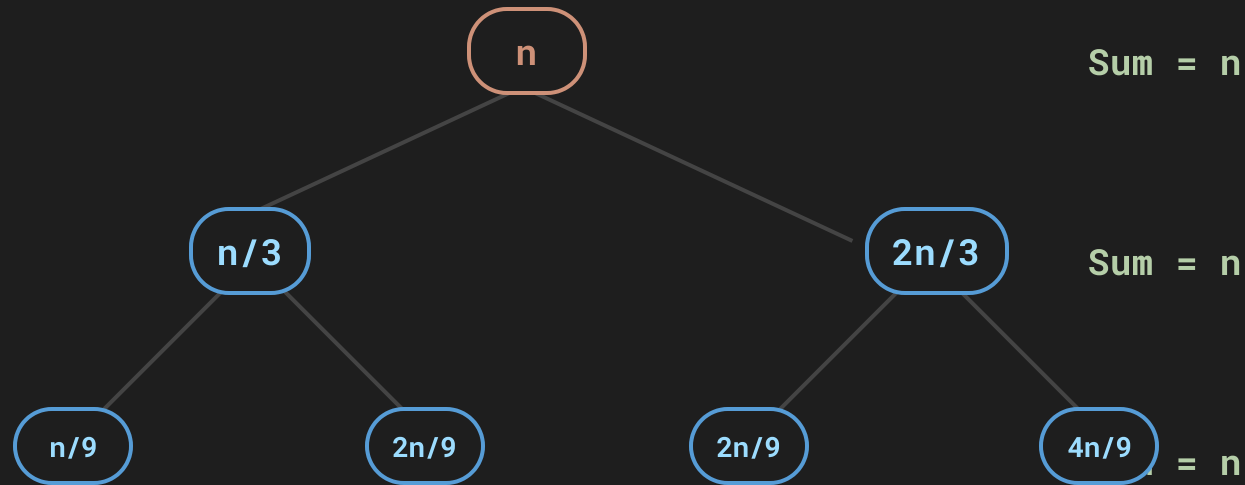
$$\text{Total} = \sum (\text{Level Costs})$$

Recursion-Tree: Simple Example

$$T(n) = 2T(n/2) + n^2$$

Recursion Tree: Complex Example

$$T(n) = T(n/3) + T(2n/3) + n$$



1. Unbalanced Tree

The tree leans to the right. The $2n/3$ branch reduces the problem size slower than the $n/3$ branch, creating a deeper tree on the right.

2. Cost Per Level

At every level (until leaves appear), the cost sums to exactly n . (e.g., $n/3 + 2n/3 = n$)

3. Total Cost

Total $\approx$ (Cost per level) $\times$ (Number of levels) $O(n \log n)$

Shortest Path: $\log_3 n$

Longest Path: $\log_{3/2} n$

Master Method: Introduction

// The "Cookbook" Approach

Standard Form

$$T(n) = aT(n/b) + f(n)$$

where $a \geq 1$, $b > 1$, and $f(n)$ is asymptotically positive

$f(n)$

Cost of the work done at the **root**
(divide & combine)

VS

$n^{\log_b a}$

Number of leaves in the recursion tree
(Watershed Function)

The Comparison Principle: The solution is determined by whichever function grows faster.

Master Method: Case 1

// The "Leaves Dominate" Case

if $f(n) = O(n^{\log_b a - \epsilon})$ // for $\epsilon > 0$

Leaves > Root

then $T(n) = \Theta(n^{\log_b a})$

Simple Example

$$T(n) = 8T(n/2) + n^2$$

$a=8, b=2$

Watershed: $n^{\log_2 8} = n^3$

Driving: $f(n) = n^2$

$n^2 < n^3$ (by factor n^1)

$$\Rightarrow T(n) = \Theta(n^3)$$

Complex Example

$$T(n) = 9T(n/3) + n$$

$a=9, b=3$

Watershed: $n^{\log_3 9} = n^2$

Driving: $f(n) = n$

$n < n^2$ (by factor n^1)

$$\Rightarrow T(n) = \Theta(n^2)$$

Master Method: Case 2

Balanced Growth



$$f(n) = \theta(n^{\log_b a})$$

The driving function $f(n)$ grows at the **same rate** as the watershed function.



$$T(n) = \theta(n^{\log_b a} \lg n)$$

Multiply the watershed function by a logarithmic factor.

Simple: Merge Sort

$$T(n) = 2T(n/2) + n$$

$$a = 2 \quad b = 2 \quad f(n) = n$$

$$n^{\log_2 2} = n^1 = f(n)$$

$$\theta(n \lg n)$$

Complex: Binary Search

$$T(n) = T(n/2) + 1$$

$$a = 1 \quad b = 2 \quad f(n) = 1$$

$$n^{\log_2 1} = n^0 = 1 = f(n)$$

$$\theta(\lg n)$$

Master Method: Case 3

// The "Root Dominates" Case

PRIMARY CONDITION

$$f(n) = \Omega(n^{\log_b a + \epsilon})$$

Driving grows polynomially faster than leaves.

$$\text{then } T(n) = \Theta(f(n))$$

Regularity Condition:

$$af(n/b) \leq cf(n)$$

for some constant $c < 1$ and large n .

Simple

$$T(n) = 2T(n/2) + n^3$$

Watershed: n 1

Driving: n 3

Compare: $n^3 > n^1$ ($\epsilon=2$)

Regularity: $2(n/2)^3 = n^3/4 \leq cn^3$

$$\Theta(n^3)$$

Complex

$$T(n) = 3T(n/4) + n \lg n$$

Watershed: n $\log_4 3 \approx n^{0.79}$

Driving: n $\lg n$

Compare: $n^1 > n^{0.79}$ ($\epsilon \approx 0.2$)

Regularity: $3(n/4 \lg(n/4)) \leq cn \lg n$

$$\Theta(n \lg n)$$

Summary & Practice

// Recap and Challenge Problems

Solutions to Practice Problems

// Master Method Application

Problem 1

CASE 1

$$T(n) = 4T(n/2) + n$$

$$a = 4, b = 2$$

$$f(n) = n$$

$$n^{\log_2 4} = n^2$$

$$n < n^2$$

(Leaves Dominate)

$$\theta(n^2)$$

Problem 2

CASE 2

$$T(n) = 4T(n/2) + n^2$$

$$a = 4, b = 2$$

$$f(n) = n^2$$

$$n^{\log_2 4} = n^2$$

$$n^2 = n^2$$

(Balanced)

$$\theta(n^2 \lg n)$$

Problem 3

CASE 3

$$T(n) = 4T(n/2) + n^3$$

$$a = 4, b = 2$$

$$f(n) = n^3$$

$$n^{\log_2 4} = n^2$$

$$n^3 > n^2$$

(Root Dominates)

$$\theta(n^3)$$